

Computer Architecture Lab Manual

Experiment 1: Data representation

Objective:

1. To illustrate the concept of data representation

Theory:

Data representation: It refers to the format of data that are stored in the memory, processed and transmitted. In computer, data are stored in digital formats that can be handled by electronic circuitry. There are various forms of representing data, some of them are:

- a) Binary: Binary number system uses 1's and 0's called bits. It is also called a positional notation. E.g.: 10011, 11000
- b) Hexadecimal:
- c) Octal:
- d) 1's and 2's complement:

Algorithm:

- i. Start
- ii. Initialize a function to find binary 1's and 2's complement, hexadecimal, octal equivalent of number
- iii. Input a number to be converted
- iv. Find the equivalent numbers
- v. Print the binary 1's and 2's complement, hexadecimal and octal equivalent numbers
- vi. Stop

Source code:

```
#include <stdio.h>
#include <conio.h>
void binary (int );
int main()
{
    int n;
    printf("enter a number");
    scanf("%d",&n);
    printf("\n the hex equivalent is %x",
n);
    printf("\n the octal equivalent is %o",
n);
    printf("\n the binary equivalent is:");
    binary(n);
    printf("\n the 1's complement is:");
    binary(-n-1);
    printf("\n the 2's complement
is:");
    binary(-n);
    return 0;
}

void binary (int a)
{
    int i, k;
    for (i=3;i>=0;i--)
    {
        k=(a>>i)&1;
        printf("%d", k);
    }
}
```

Using bitwise operator

```
#include <stdio.h>
int onesComplement(int num) {
    int numBits = 4;                // Assuming 4-bit binary numbers for
    simplicity

    for (int i = 0; i < numBits; i++) {
        num = num ^ (1 << i);        // Bitwise inverting each bit
    }
    return num;
}

int main() {
    char binaryNumber[5];            // 5 bits + null terminator

    printf("Enter an 4-bit binary number: ");
    scanf("%s", binaryNumber);

    // Convert the binary string to an integer
    int decimalNumber = 0;
    for (int i = 0; binaryNumber[i] != '\0'; i++) {
        decimalNumber = decimalNumber * 2 + (binaryNumber[i] - '0');
    }
    printf("Original binary number: %s\n", binaryNumber);
    int onesComp = onesComplement(decimalNumber);
    int twosComp = onesComp + 1;
    printf("1's complement: ");
    // Displaying the result as a binary number
    for (int i = 3; i >= 0; i--) {
        printf("%d", (onesComp >> i) & 1);
    }
    printf("\n");
    printf("2's complement: ");
    for (int i = 3; i >= 0; i--) {
        printf("%d", (twosComp >> i) & 1);
    }
    printf("\n");
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
```

```
void twosComplement(char* binary) {
    int length = strlen(binary);
    int i;
```

```

// Find the first '1' from the right side
int firstOneIndex = -1;
for (i = length - 1; i >= 0; i--) {
    if (binary[i] == '1') {
        firstOneIndex = i;
        break;
    }
}

// If no '1' is found, return as there's no need to flip any bits
if (firstOneIndex == -1) {
    return;
}

// Flip all bits (including the rightmost '1')
for (i = firstOneIndex-1; i >= 0; i--) {
    binary[i] = (binary[i] == '0') ? '1' : '0';
}

}

int main() {
    char binary[] = "1010110"; // Example binary string
    printf("Original: %s\n", binary);

    twosComplement(binary);

    printf("Twos complement: %s\n", binary);
    return 0;
}

```

Experiment 2: Data overflow

Objective:

1. To understand the concept of data overflow

Theory:

In computer programming, an integer overflow occurs when an arithmetic operation attempts to create a numeric value that is not in the range that can be represented with a given number of digits- either too larger than the maximum or lower than the minimum representable value.

Condition for overflow:

```
((AS==BS) &(AS==RS) || (AS!=BS))
{
    NO overflow, display result;
}
```

Algorithm:

- i. Start
- ii. Observe carry into the sign bit position & carry out the sign bit position
- iii. If the two carry aren't equal, overflow should be detected
- iv. If the two carry are applied to an X-OR gate overflow will be detected when output of gate is 1.
- v. Stop

Source code:

```
#include <stdio.h>
int main()
{
    int a,b,r,as,bs,rs;
    printf("enter the two numbers a and b");
    scanf("%d%d", &a,&b);
    r=a+b;
    as=(a>>3)&1;
    bs=(b>>3)&1;
    rs=(r>>3)&1;
    if(((as==bs)&&(as==rs))||(as!=bs))
    {
        printf("\nresult=%d",r);
    }
    else
    {
        printf("\n overflow detected\t the result is:%d",r);
    }
}
```

Experiment 3: Introduction to VHDL

Objective:

1. To implement the Basic gates and universal gates using VHDL

Theory:

VHDL (VHSIC Hardware Description Language) is a hardware description language used in electronic design automation to describe digital and mixed-signal systems such as field-programmable gate arrays and integrated circuits. VHDL can also be used as a general purpose parallel programming language.

VHDL is commonly used to write text models that describe a logic circuit. Such a model is processed by a synthesis program, only if it is part of the logic design. A simulation program is used to test the logic design using simulation models to represent the logic circuits that interface to the design. This collection of simulation models is commonly called a **testbench**.

A VHDL simulator is typically an event-driven simulator. The simulation alters between two modes: statement execution, where triggered statements are evaluated, and event processing, where events in the queue are processed.

VHDL has constructs to handle the parallelism inherent in hardware designs, but these constructs (*processes*) differ in syntax from the parallel constructs in Ada (*tasks*). Like Ada, VHDL is strongly typed and is not case sensitive.

In VHDL, entity is used to describe a hardware module. An entity can be described using:

- i. Entity declaration
- ii. Architecture
- iii. Configuration
- iv. Package declaration
- v. Package body

Entity Declaration:

It defines the name, input/output signals and modes of hardware module.

Syntax:

```
entity entity_name is
    port declaration;
end entity_name;
```

Architecture:

Can be described using structural, data flow, behavior or mixed type.

Syntax:

```
architecture architecture_name of entity_name is
    architecture architecture_declarative part;
begin
    statements;
end architecture_name;
logic operation: NAND gate
```

Architecture Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity And_gate is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          y : out STD_LOGIC);
end And_gate;

architecture Behavioral of And_gate is
begin
    y <= A and B;

end Behavioral;
```

Experiment 4: Half adder**Objective:**

1. To implement the Half adder using VHDL

Theory: (Write yourself)**Architecture:**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
    port(  A,B: in STD_LOGIC;
          S: out STD_LOGIC;
          Cout: out STD_LOGIC
        );
end half_adder;

architecture Behavioral of half_adder is
begin
    S <= A xor B;
    Cout<= A and B;
end Behavioral;
```

Experiment 4: 4-bit parallel adder**Objective:**

2. To implement the 4-bit parallel adder using VHDL

Theory:

Parallel adder is a digital circuit capable of finding the arithmetic sum of two binary numbers that is greater than one bit in length by operating on corresponding pairs of bits in parallel. It consists of full adder connected in a chain where the output carry from each full adder is connected to the carry input of the next higher order full adder in a chain. A n-bit parallel adder requires n-full adder to perform the operation.

Architecture:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity parallel_adder is
port(  A,B: in STD_LOGIC_VECTOR(3 downto 0);
      S: out STD_LOGIC_VECTOR(3 downto 0);
      Cin: in STD_LOGIC;
      Cout: out STD_LOGIC
      );
end parallel_adder;

architecture Behavioral of parallel_adder is
begin
process (A,B,Cin)
variable t: STD_LOGIC;
begin
t :=Cin;
for i in 0 to 3 loop
S(i)<=A(i) Xor B(i) Xor t;
t := (A(i) and B(i)) or (t and A(i) or (t and B(i)));
end loop;
Cout<=t;
end process;
end Behavioral;
```

Experiment 5: 3-Segment Pipeline

Objective:

1. To implement 3-Segment Pipeline using VHDL

Theory:

$$Y = (a+b)*c$$

Architecture:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

use IEEE.NUMERIC_STD.ALL;

entity pipeline is
  Port ( a : in integer;
        b : in integer;
        c : in integer;
        clk : in STD_LOGIC;
        y : out integer);
end pipeline;

architecture Behavioral of pipeline is
  signal r1,r2,r3,r4 : integer :=0;
begin
  y<=r6;
  process(clk)
  begin
    if(rising_edge(clk)) then
      for i in 0 to 2 loop
        case(i) is
          when 0=>
            r1<=a;
            r2<=b;
            r3<=c;
          when 1=>
            r4<=r1+r2;
          when 2=>
            r5<=r4*r3;
          when others=>
            null;
          end case;
        end loop;
      end if;
    end process;
  end Behavioral;

```

Experiment 6: ALU Design

Objective:

1. To implement ALU

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.NUMERIC_STD.all;
entity ALU is
  generic ( constant N: natural:=1); -- number of shifted or rotated bits
  Port (
    A, B: in STD_LOGIC_VECTOR (7 downto 0); -- 2 inputs 8-bit

```

```

        ALU_Sel: in STD_LOGIC_VECTOR (3 downto 0); -- 1 input 4-bit for
selecting function
        ALU_Out: out STD_LOGIC_VECTOR (7 downto 0); -- 1 output 8-bit
        Carryout: out std_logic      -- Carryout flag
    );
end ALU;
architecture Behavioral of ALU is
    signal ALU_Result: std_logic_vector (7 downto 0);
    signal tmp: std_logic_vector (8 downto 0);

begin
    process(A,B,ALU_Sel)
    begin
        case(ALU_Sel) is
            when "0000" => -- Addition
                ALU_Result <= A + B ;
            when "0001" => -- Subtraction
                ALU_Result <= A - B ;
            when "0010" => -- Multiplication
                ALU_Result<=std_logic_vector(to_unsigned((to_integer(unsigned(A)) *
to_integer(unsigned(B))),8)) ;
            when "0011" => -- Division
                ALU_Result <= std_logic_vector(to_unsigned(to_integer(unsigned(A)) /
to_integer(unsigned(B))),8)) ;
            when "0100" => -- Logical shift left
                ALU_Result <= std_logic_vector(unsigned(A) sll N);
            when "0101" => -- Logical shift right
                ALU_Result <= std_logic_vector(unsigned(A) srl N);
            when "0110" => -- Rotate left
                ALU_Result <= std_logic_vector(unsigned(A) rol N);
            when "0111" => -- Rotate right
                ALU_Result <= std_logic_vector(unsigned(A) ror N);
            when "1000" => -- Logical and
                ALU_Result <= A and B;
            when "1001" => -- Logical or
                ALU_Result <= A or B;
            when "1010" => -- Logical xor
                ALU_Result <= A xor B;
            when "1011" => -- Logical nor
                ALU_Result <= A nor B;
            when "1100" => -- Logical nand
                ALU_Result <= A nand B;
            when "1101" => -- Logical xnor
                ALU_Result <= A xnor B;
            when "1110" => -- Greater comparison
                if(A>B) then
                    ALU_Result <= x"01" ;
                else
                    ALU_Result <= x"00" ;
                end if;
        end case;
    end process;
end ALU;

```

```

when "1111" => -- Equal comparison
    if(A=B) then
        ALU_Result <= x"01" ;
    else
        ALU_Result <= x"00" ;
    end if;
when others => ALU_Result <= A + B ;
end case;
end process;
    ALU_Out <= ALU_Result; -- ALU out
    tmp <= ('0' & A) + ('0' & B);
    Carryout <= tmp(8); -- Carryout flag
end Behavioral;

```